123

Reinforcement Learning I

Instructor: Hyunwoo J. Kim

Question/Discussion/Recap

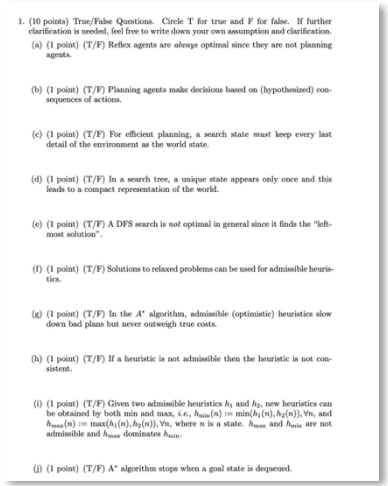
Past midterm exams? Exercises?

123

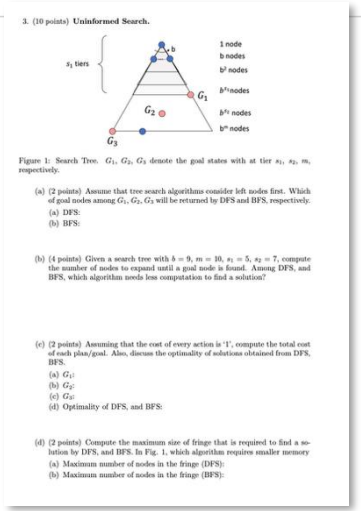
7주차 2026.04.13 ~ 2026.04.19

Coding Assignment 3 due by 04/30 2026-04-13 00:00 ~ 2026-04-30 23:59

Midterm Prep: Example Questions



1. (10 points) True/False Questions. Circle T for true and F for false. If further clarification is needed, feel free to write down your own assumption and clarification.
   (a) (1 point) (T/F) Reflex agents are *always* optimal since they are not planning agents.
   (b) (1 point) (T/F) Planning agents make decisions based on (hypothesized) consequences of actions.
   (c) (1 point) (T/F) For efficient planning, a search state *must* keep every last detail of the environment as the world state.
   (d) (1 point) (T/F) In a search tree, a unique state appears only once and this leads to a compact representation of the world.
   (e) (1 point) (T/F) A DFS search is *not* optimal in general since it finds the "left-most solution".
   (f) (1 point) (T/F) Solutions to relaxed problems can be used for admissible heuristics.
   (g) (1 point) (T/F) In the A^* algorithm, admissible (optimistic) heuristics slow down bad plans but never outweigh true costs.
   (h) (1 point) (T/F) If a heuristic is not admissible then the heuristic is not consistent.
   (i) (1 point) (T/F) Given two admissible heuristics h_1 and h_2, new heuristics can be obtained by both min and max, i.e., $h_{\min}(n) := \min(h_1(n), h_2(n)), \forall n$, and $h_{\max}(n) := \max(h_1(n), h_2(n)), \forall n$, where n is a state. $h_{\max}$ and $h_{\min}$ are not admissible and $h_{\max}$ dominates $h_{\min}$.
   (j) (1 point) (T/F) A* algorithm stops when a goal state is dequeued.



3. (10 points) **Uninformed Search.**

Figure 1: Search Tree. G_1, G_2, G_3 denote the goal states with at tier s_1, s_2, m, respectively.

(a) (2 points) Assume that tree search algorithms consider left nodes first. Which of goal nodes among G_1, G_2, G_3 will be returned by DFS and BFS, respectively.
   (a) DFS:
   (b) BFS:

(b) (4 points) Given a search tree with $b = 9$, $m = 10$, $s_1 = 5$, $s_2 = 7$, compute the number of nodes to expand until a goal node is found. Among DFS, and BFS, which algorithm needs less computation to find a solution?

(c) (2 points) Assuming that the cost of every action is '1', compute the total cost of each plan/goal. Also, discuss the optimality of solutions obtained from DFS, BFS.
   (a) G_1:
   (b) G_2:
   (c) G_3:
   (d) Optimality of DFS, and BFS:

(d) (2 points) Compute the maximum size of fringe that is required to find a solution by DFS, and BFS. In Fig. 1, which algorithm requires smaller memory
   (a) Maximum number of nodes in the fringe (DFS):
   (b) Maximum number of nodes in the fringe (BFS):

Slide credit: CS188 of Klein and Abbeel

Midterm exam

123

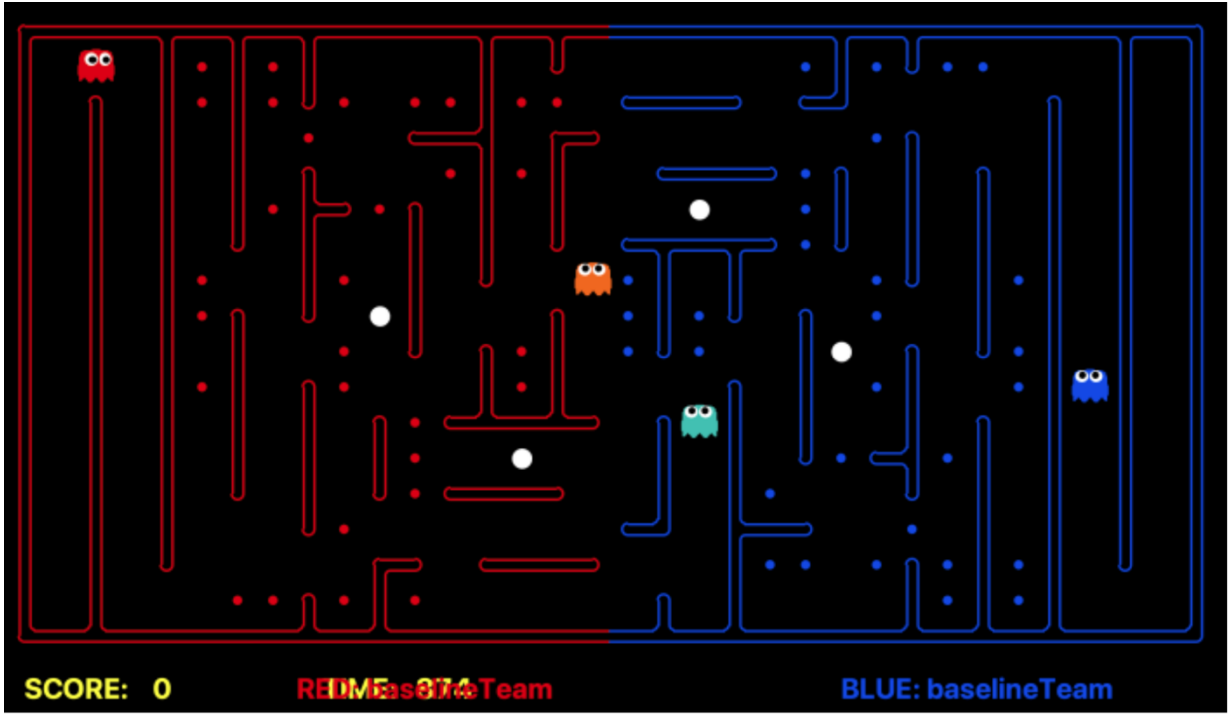
- 04/23 (Thu) 9:30 AM ~ 11:30AM
- The midterm exam will be a completely closed-book exam.
- No cheatsheets or notes are allowed.
- No electronic devices may be used during the exam.

Slide credit: CS188 of Klein and Abbeel

PACMAN League due by 4/30

123

This mini-contest involves a multi-player capture-the-flag variant of Pacman, where agents control both Pacman and ghosts in coordinated team-based strategies. Your team will try to eat the food on the far side of the map, while defending the food on your home side.



Extra Credit (30pts)

Students who perform well in the **final** submission will receive the following extra credit:

- 1st to 5th place: 30 points (05/07 presentation)
- 6th to 10th place: 20 points
- 11th to 15th place: 15 points
- 16th to 20th place: 10 points
- 21st to 25th place: 5 points

Today's agenda

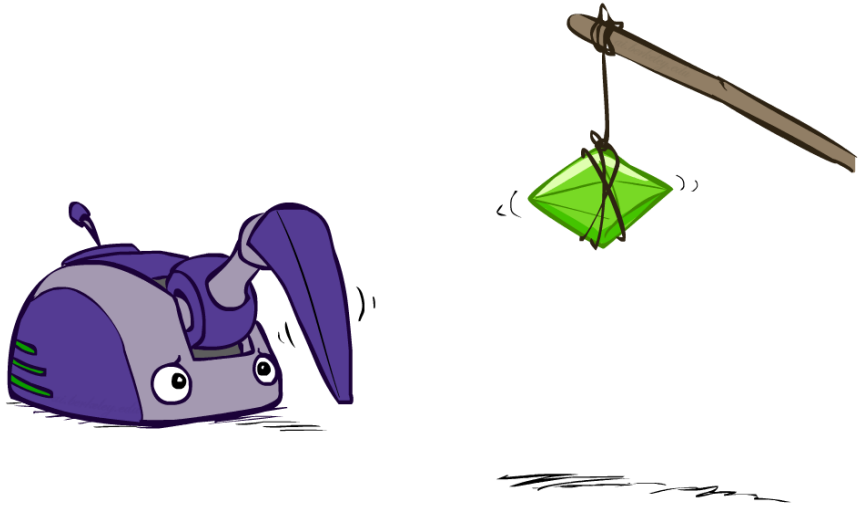
123

- Reinforcement Learning
  - Model-based Learning
  - Model-free Learning
  - Temporal Difference Learning
  - Active Reinforcement Learning / Q-Learning

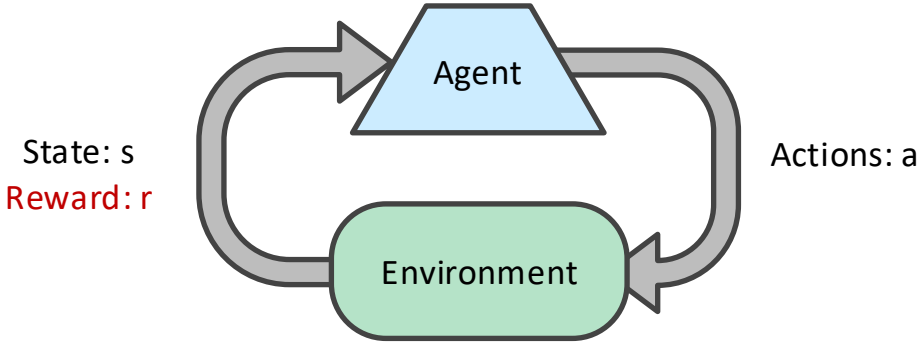
Slide credit: CS188 of Klein and Abbeel

Reinforcement Learning

123



Reinforcement Learning



- Basic idea:
  - Receive feedback in the form of rewards
  - Agent's utility is defined by the reward function
  - Must (learn to) act so as to maximize expected rewards
  - All learning is based on observed samples of outcomes!

Slide credit: CS188 of Klein and Abbeel

Example: Learning to Walk

123

Initial

A Learning Trial

After Learning [1K Trials]

[Kohl and Stone, ICRA 2004]

Slide credit: CS188 of Klein and Abbeel

Example: Learning to Walk

Initial [Video: AIBO WALK – initial]

[Kohl and Stone, ICRA 2004]

Example: Learning to Walk

Training [Video: AIBO WALK – training]

[Kohl and Stone, ICRA 2004]

Slide credit: CS188 of Klein and Abbeel

Example: Learning to Walk

Finished [Video: AIBO WALK – finished]

[Kohl and Stone, ICRA 2004]

Slide credit: CS188 of Klein and Abbeel

Deep Q-learning by Google DeepMind

https://www.youtube.com/watch?v=V1eYniJ0Rnk

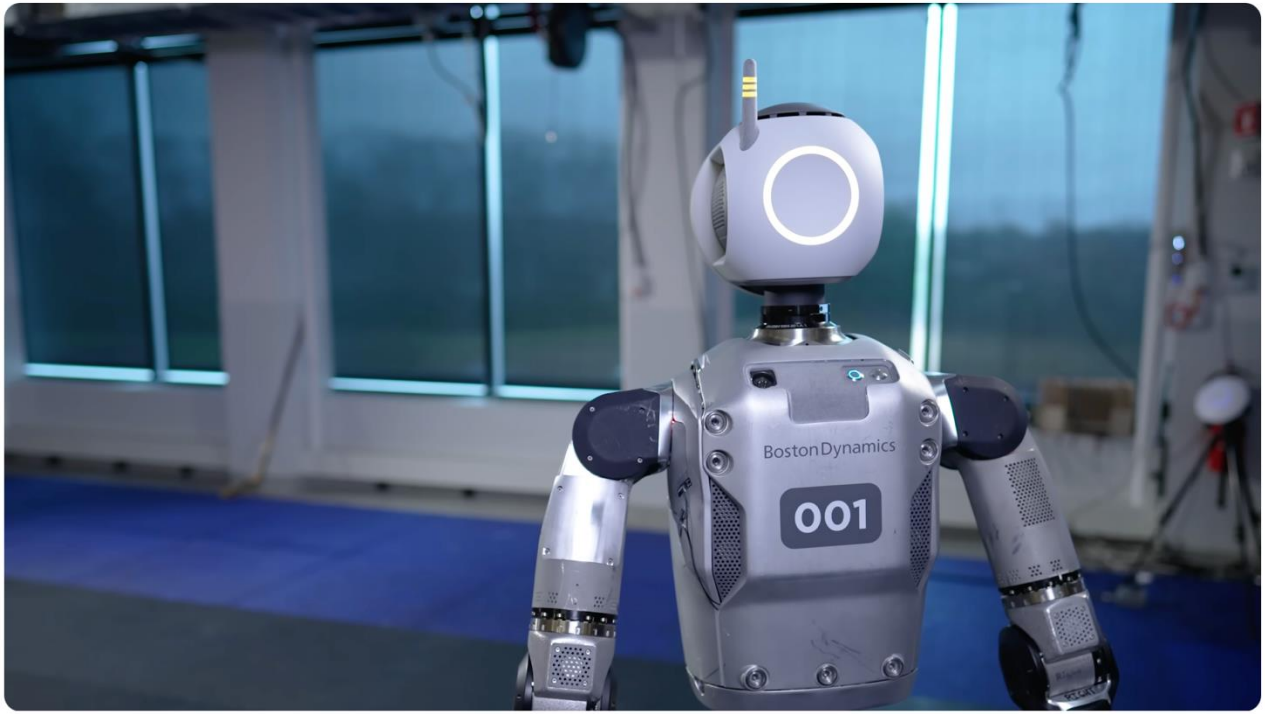
Boston Dynamics

123

https://www.youtube.com/watch?v=hSjKoEva5bg

Boston Dynamics

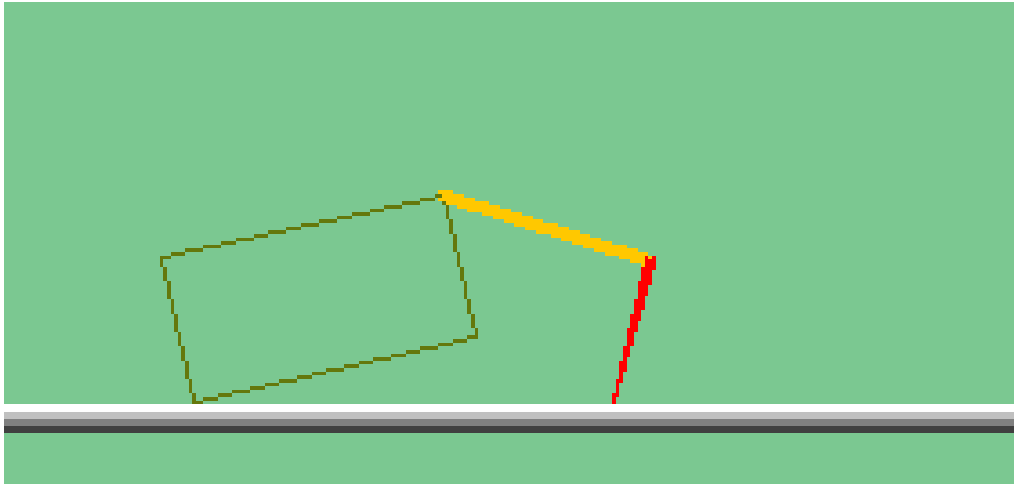
123



https://www.youtube.com/watch?v=29ECwExc-_M

The Crawler!

123



[Demo: Crawler Bot (L10D1)]

Slide credit: CS188 of Klein and Abbeel

Video of Demo Crawler Bot

123

Slide credit: CS188 of Klein and Abbeel

Reinforcement Learning

123

- Still assume a Markov decision process (MDP):
  - A set of states $s \in S$
  - A set of actions (per state) A
  - A model T(s,a,s')
  - A reward function R(s,a,s')
- Still looking for a policy $\pi(s)$
- New twist: don't know T or R
  - I.e. we don't know which states are good or what the actions do
  - Must actually try actions and states out to learn

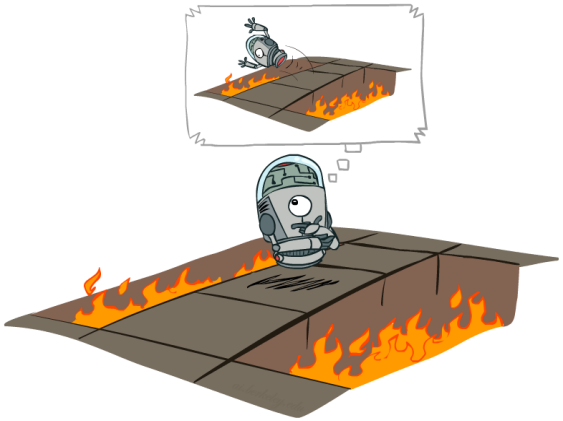

Cool


Warm


Overheated

Slide credit: CS188 of Klein and Abbeel

Offline (MDPs) vs. Online (RL)



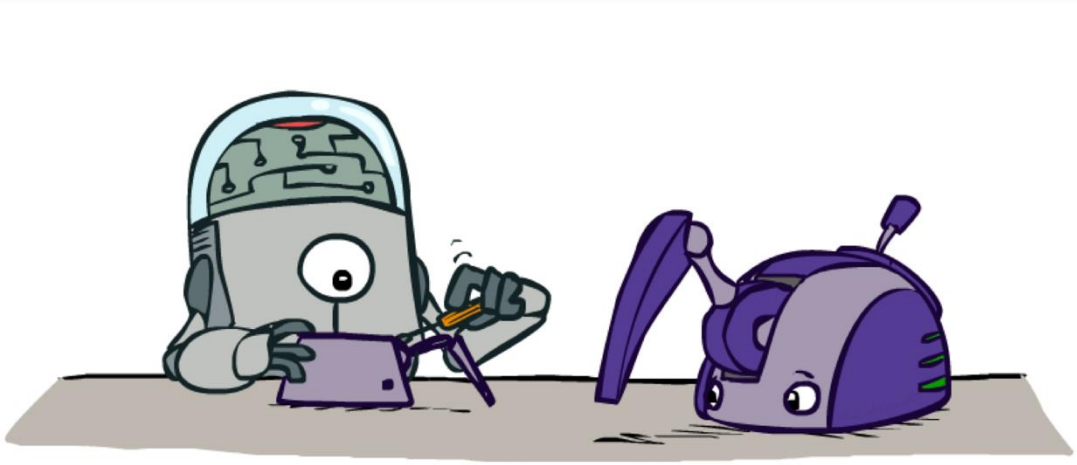
Offline Solution

Online Learning

Slide credit: CS188 of Klein and Abbeel

Model-Based Learning

123



Slide credit: CS188 of Klein and Abbeel

Model-Based Learning

123

- Model-Based Idea:
  - Learn an approximate model based on experiences
  - Solve for values as if the learned model were correct
- Step 1: Learn empirical MDP model
  - Count outcomes s' for each s, a
  - Normalize to give an estimate of $\hat{T}(s, a, s')$
  - Discover each $\hat{R}(s, a, s')$ when we experience (s, a, s')
- Step 2: Solve the learned MDP
  - For example, use value iteration, as before

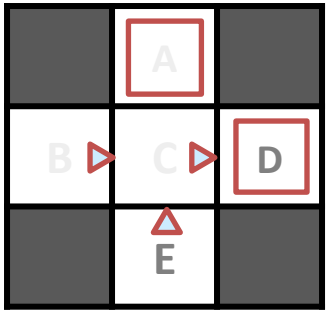
Slide credit: CS188 of Klein and Abbeel

Example: Model-Based Learning

123

Input Policy

π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Learned Model

$\hat{T}(s, a, s')$

T(B, east, C) =
T(C, east, D) =
T(C, east, A) =
...

$\hat{R}(s, a, s')$

R(B, east, C) =
R(C, east, D) =
R(D, exit, x) =
...

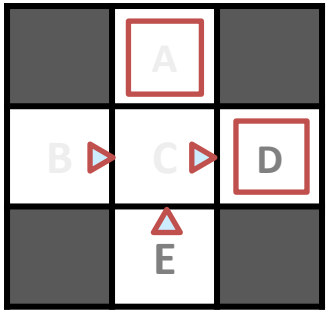
Slide credit: CS188 of Klein and Abbeel

Example: Model-Based Learning

123

Input Policy

π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Learned Model

$\hat{T}(s, a, s')$

T(B, east, C) = 1.00
T(C, east, D) = 0.75
T(C, east, A) = 0.25
...

$\hat{R}(s, a, s')$

R(B, east, C) = -1
R(C, east, D) = -1
R(D, exit, x) = +10
...

Slide credit: CS188 of Klein and Abbeel

Example: Expected Age

Goal: Compute expected age of **CS470** students

Known P(A)

$$E[A] = \sum_a P(a) \cdot a \quad = 0.35 \times 20 + \ldots$$

Without P(A), instead collect samples $[a_1, a_2, \ldots a_N]$

Unknown P(A): "Model Based"

$$\hat{P}(a) = \frac{\text{num}(a)}{N}$$

$$E[A] \approx \sum_a \hat{P}(a) \cdot a$$

Why does this work? Because eventually you learn the right model.

Unknown P(A): "Model Free"

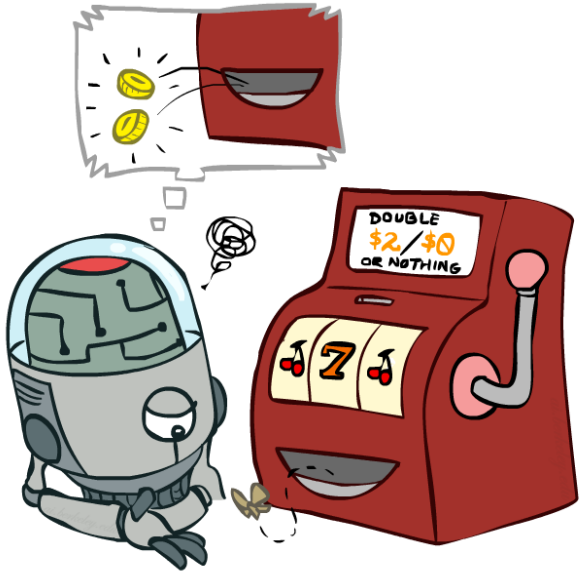
$$E[A] \approx \frac{1}{N} \sum_i a_i$$

Why does this work? Because samples appear with the right frequencies.

Slide credit: CS188 of Klein and Abbeel

Model-Free Learning

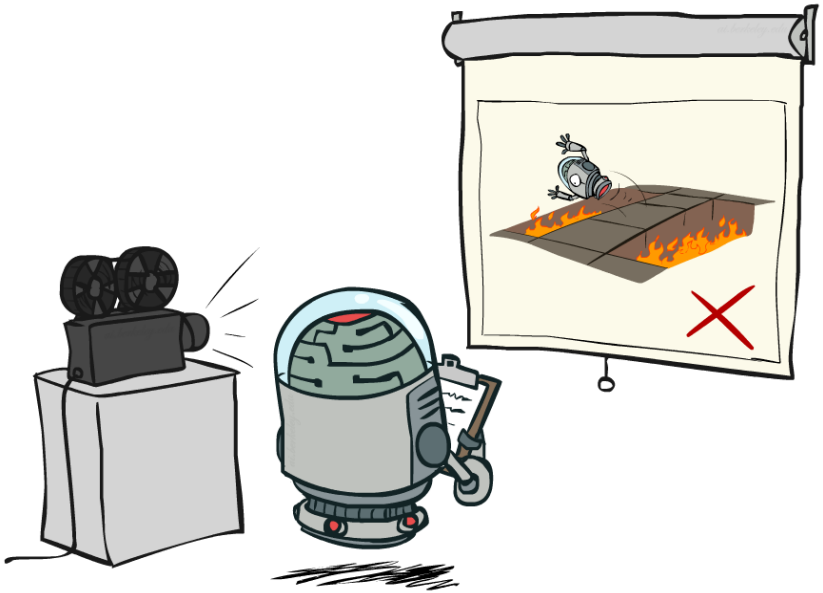
123



Slide credit: CS188 of Klein and Abbeel

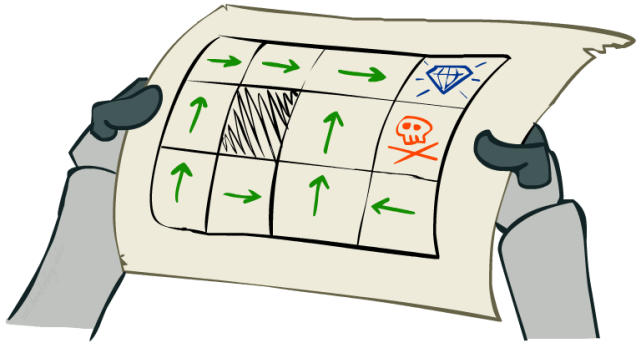
Passive Reinforcement Learning

123



Passive Reinforcement Learning

- Simplified task: policy evaluation
  - Input: a fixed policy $\pi(s)$
  - You don't know the transitions T(s,a,s')
  - You don't know the rewards R(s,a,s')
  - Goal: learn the state values



- In this case:
  - Learner is "along for the ride"
  - No choice about what actions to take
  - Just execute the policy and learn from experience
  - This is NOT offline planning! You actually take actions in the world.

Slide credit: CS188 of Klein and Abbeel

Direct Evaluation

123

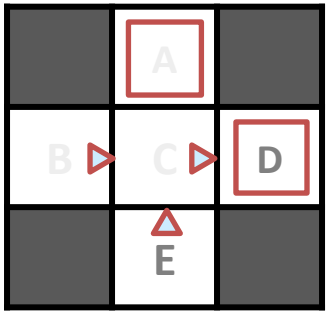
- Goal: Compute values for each state under π
- Idea: Average together observed sample values
  - Act according to π
  - Every time you visit a state, write down what the sum of discounted rewards turned out to be
  - Average those samples
- This is called direct evaluation

Slide credit: CS188 of Klein and Abbeel

Example: Direct Evaluation

123

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

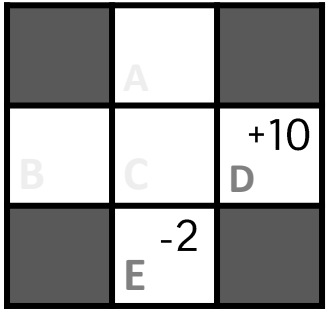
Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

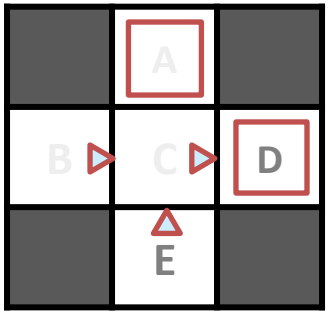
Output Values



Example: Direct Evaluation

123

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

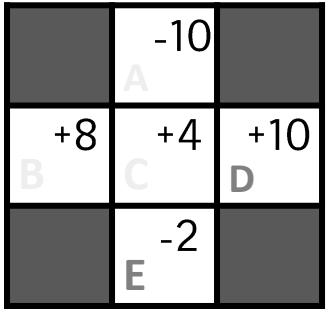
Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Output Values

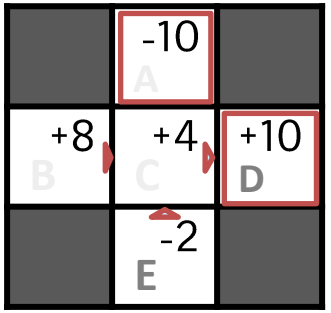


Slide credit: CS188 of Klein and Abbeel

Problems with Direct Evaluation

- What's good about direct evaluation?
  - It's easy to understand
  - It doesn't require any knowledge of T, R
  - It eventually computes the correct average values, using just sample transitions
- What bad about it?
  - It wastes information about state connections
  - Each state must be learned separately
  - So, it takes a long time to learn

Output Values



If B and E both go to C under this policy, how can their values be different?

Slide credit: CS188 of Klein and Abbeel

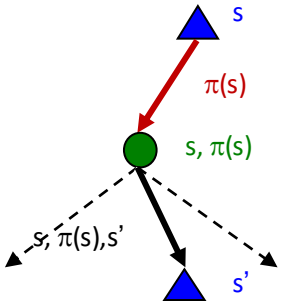
Why Not Use Policy Evaluation?

123

- Simplified Bellman updates calculate V for a fixed policy:
  - Each round, replace V with a one-step-look-ahead layer over V

$$V_0^\pi(s) = 0$$

$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$



  - This approach fully exploited the connections between the states
  - Unfortunately, we need T and R to do it!
- Key question: how can we do this update to V without knowing T and R?
  - In other words, how to we take a weighted average without knowing the weights?

Slide credit: CS188 of Klein and Abbeel

Sample-Based Policy Evaluation?

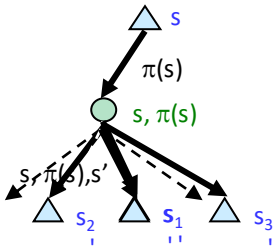
- We want to improve our estimate of V by computing these averages:

$$V^{\pi}_{k+1}(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V^{\pi}_{k}(s')]$$

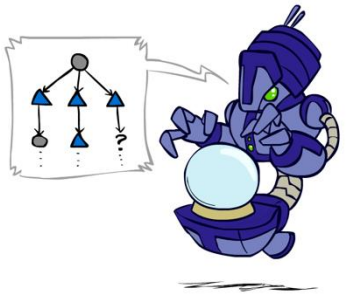
- Idea: Take samples of outcomes s' (by doing the action!) and average

$$sample_1 = R(s, \pi(s), s'_1) + \gamma V^{\pi}_{k}(s'_1)$$
$$sample_2 = R(s, \pi(s), s'_2) + \gamma V^{\pi}_{k}(s'_2)$$
$$\cdots$$
$$sample_n = R(s, \pi(s), s'_n) + \gamma V^{\pi}_{k}(s'_n)$$

$$V^{\pi}_{k+1}(s) \leftarrow \frac{1}{n} \sum_{i} sample_i$$



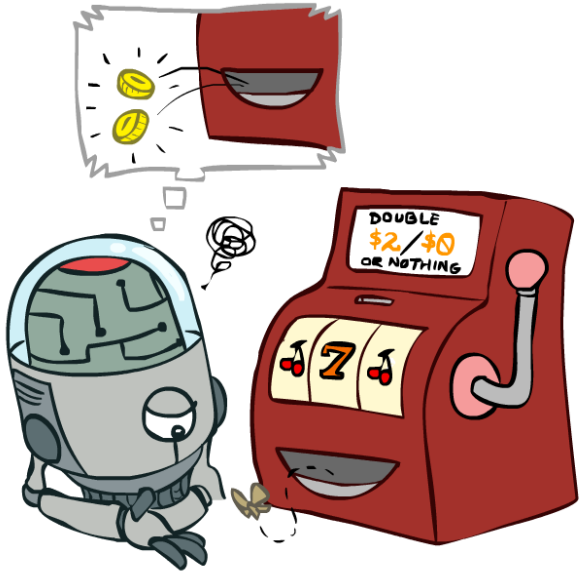
Almost! But we can't rewind time to get sample after sample from state s.



Slide credit: CS188 of Klein and Abbeel

Temporal Difference Learning

123

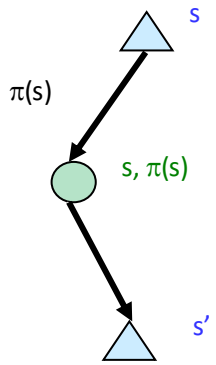


Slide credit: CS188 of Klein and Abbeel

Temporal Difference Learning

123

- Big idea: learn from every experience!
  - Update V(s) each time we experience a transition (s, a, s', r)
  - Likely outcomes s' will contribute updates more often
- Temporal difference learning of values
  - Policy still fixed, still doing evaluation!
  - Move values toward value of whatever successor occurs: running average



Sample of V(s): $sample = R(s, \pi(s), s') + \gamma V^{\pi}(s')$

Update to V(s): $V^{\pi}(s) \leftarrow (1-\alpha)V^{\pi}(s) + (\alpha)sample$

Same update: $V^{\pi}(s) \leftarrow V^{\pi}(s) + \alpha(sample - V^{\pi}(s))$

Slide credit: CS188 of Klein and Abbeel

Exponential Moving Average

123

- Exponential moving average
  - The running interpolation update: $\bar{x}_n = (1-\alpha) \cdot \bar{x}_{n-1} + \alpha \cdot x_n$
  - Makes recent samples more important:

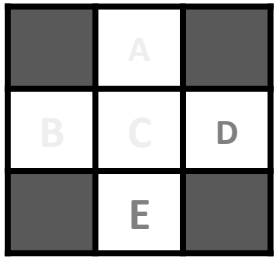
$$\bar{x}_n = \frac{x_n + (1-\alpha) \cdot x_{n-1} + (1-\alpha)^2 \cdot x_{n-2} + \dots}{1 + (1-\alpha) + (1-\alpha)^2 + \dots}$$

  - Forgets about the past (distant past values were wrong anyway)
- Decreasing learning rate (alpha) can give converging averages

Slide credit: CS188 of Klein and Abbeel

Example: Temporal Difference Learning

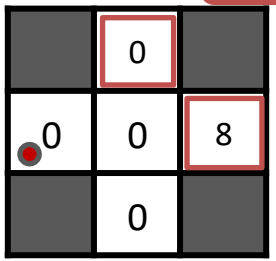
States



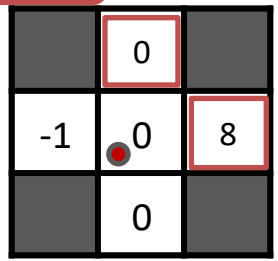
Assume: $\gamma = 1$, $\alpha = 1/2$

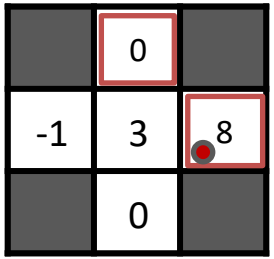
Observed Transitions

B, east, C, -2



C, east, D, -2





$$V^{\pi}(s) \leftarrow (1-\alpha)V^{\pi}(s) + \alpha\left[R(s,\pi(s),s') + \gamma V^{\pi}(s')\right]$$

Slide credit: CS188 of Klein and Abbeel

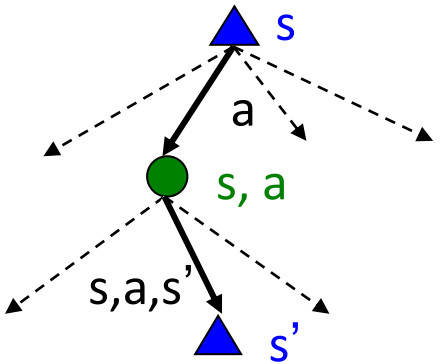
Problems with TD Value Learning

- TD value leaning is a model-free way to do policy evaluation, mimicking Bellman updates with running sample averages
- However, if we want to turn values into a (new) policy, we're sunk:

$$\pi(s) = \arg\max_a Q(s,a)$$

$$Q(s,a) = \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V(s') \right]$$

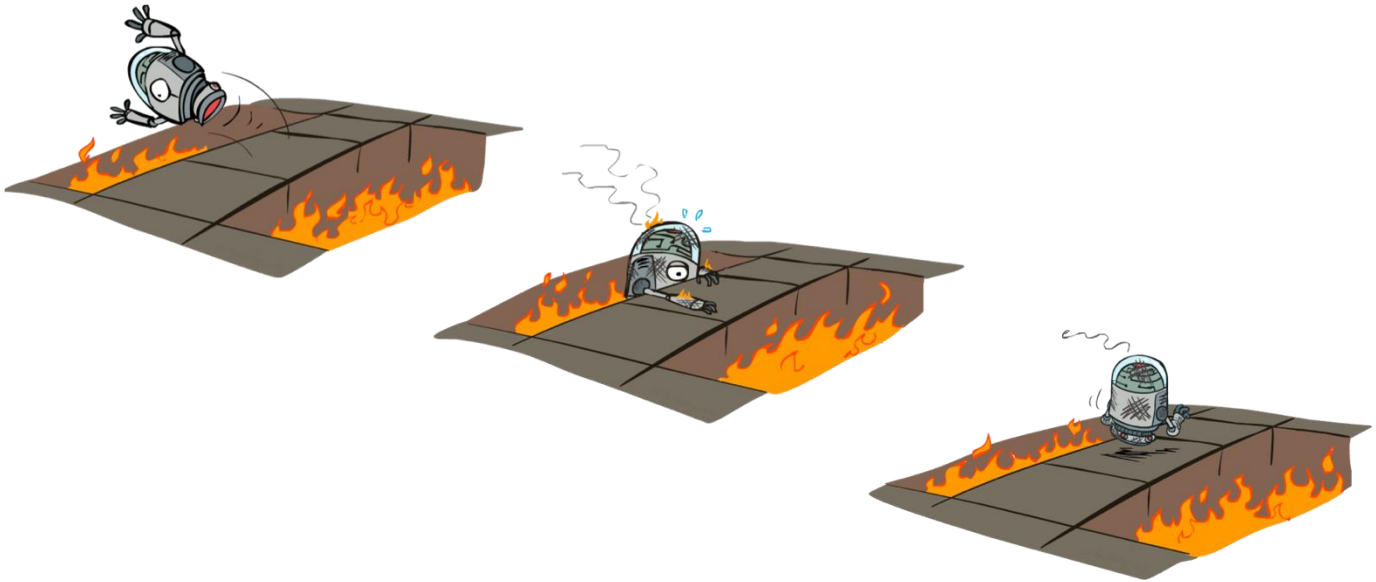
- Idea: learn Q-values, not values
- Makes action selection model-free too!



Slide credit: CS188 of Klein and Abbeel

Active Reinforcement Learning

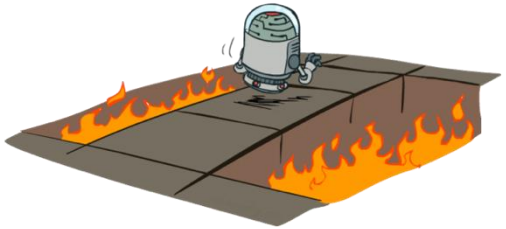
123



Active Reinforcement Learning

123

- Full reinforcement learning: optimal policies (like value iteration)
  - You don't know the transitions T(s,a,s')
  - You don't know the rewards R(s,a,s')
  - You choose the actions now
  - Goal: learn the optimal policy / values
- In this case:
  - Learner makes choices!
  - Fundamental tradeoff: exploration vs. exploitation
  - This is NOT offline planning! You actually take actions in the world and find out what happens...



Slide credit: CS188 of Klein and Abbeel

Detour: Q-Value Iteration

123

- Value iteration: find successive (depth-limited) values
  - Start with $V_0(s) = 0$, which we know is right
  - Given V_k, calculate the depth k+1 values for all states:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s') \right]$$

- But Q-values are more useful, so compute them instead
  - Start with $Q_0(s,a) = 0$, which we know is right
  - Given Q_k, calculate the depth k+1 q-values for all q-states:

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a') \right]$$

Slide credit: CS188 of Klein and Abbeel

Q-Learning

- Q-Learning: sample-based Q-value iteration

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a') \right]$$

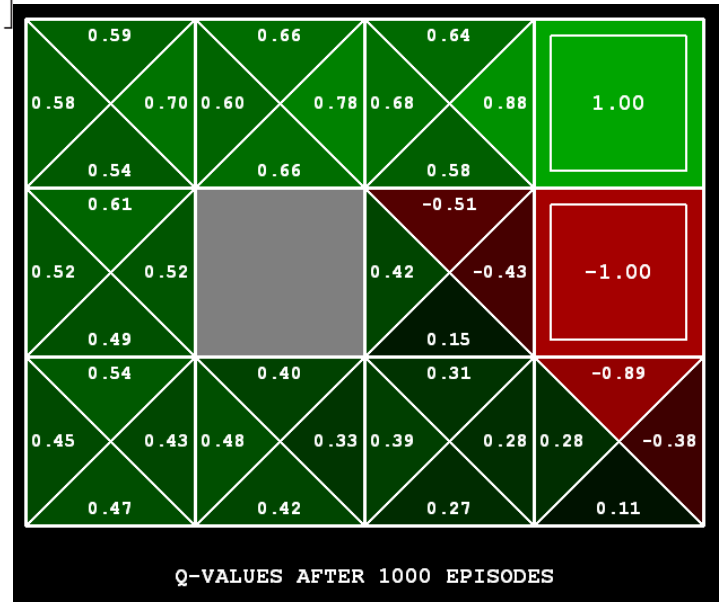
- Learn Q(s,a) values as you go
  - Receive a sample (s,a,s',r)
  - Consider your old estimate: $Q(s,a)$
  - Consider your new sample estimate:

$$sample = R(s,a,s') + \gamma \max_{a'} Q(s',a')$$

  - Incorporate the new estimate into a running average:

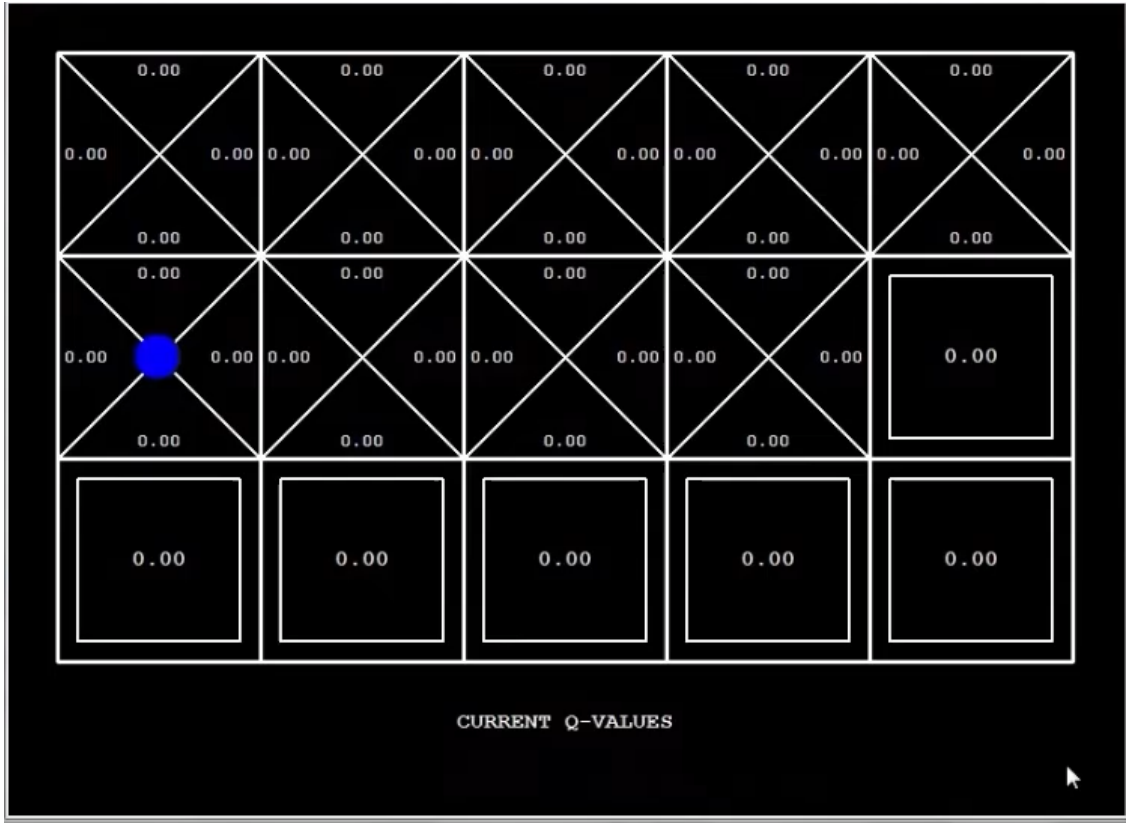
$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + (\alpha)\,[sample]$$

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$



Slide credit: CS188 of Klein and Abbeel

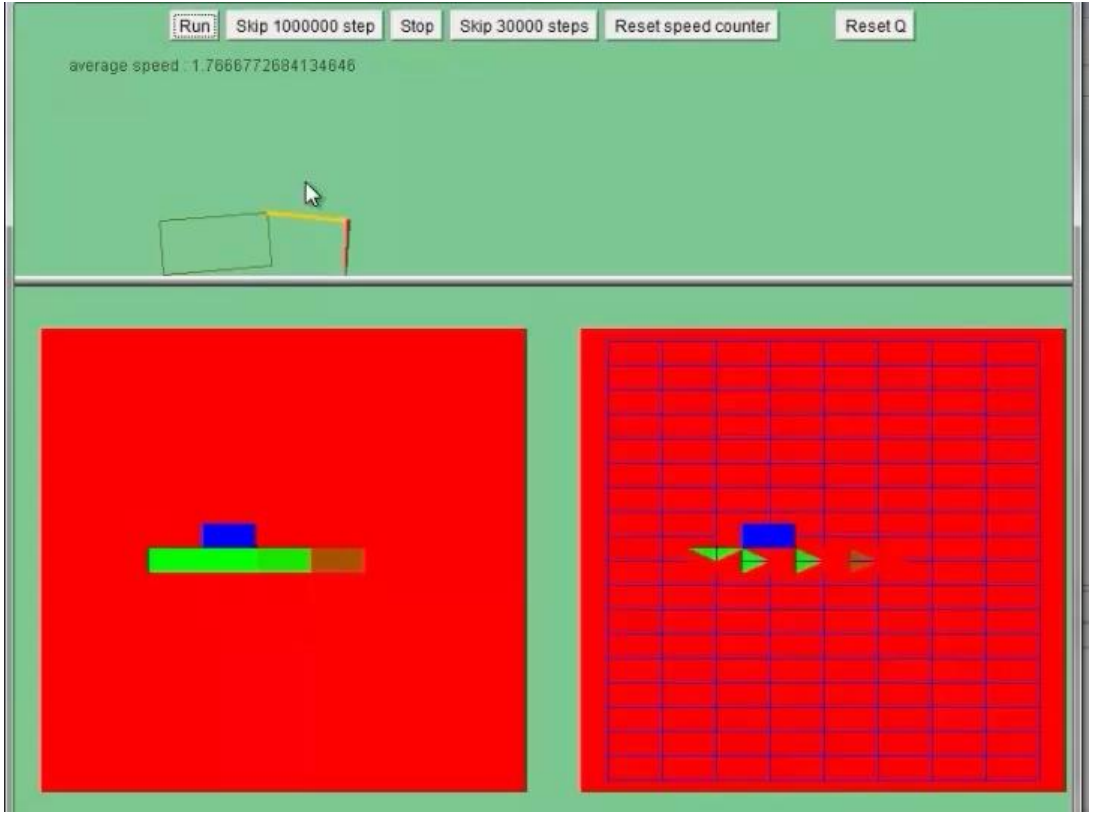
Video of Demo Q-Learning -- Gridworld



Slide credit: CS188 of Klein and Abbeel

Video of Demo Q-Learning -- Crawler

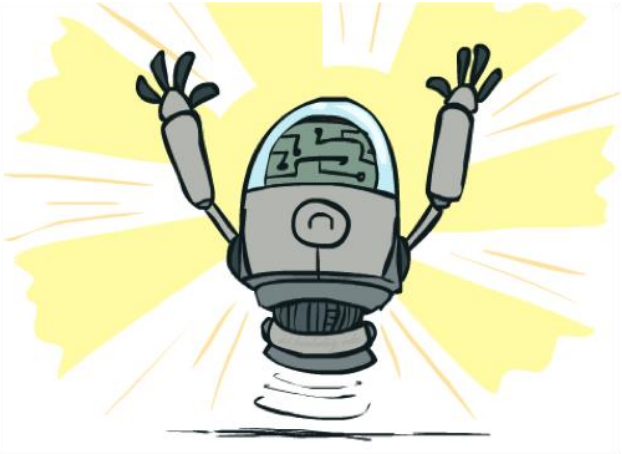
123



Slide credit: CS188 of Klein and Abbeel

Q-Learning Properties

- Amazing result: Q-learning converges to optimal policy -- even if you're acting suboptimally!
- This is called off-policy learning
- Caveats:
  - You have to explore enough
  - You have to eventually make the learning rate small enough
  - … but not decrease it too quickly
  - Basically, in the limit, it doesn't matter how you select actions (!)



Slide credit: CS188 of Klein and Abbeel

Questions

123

1. What are the two steps of model-based RL, and why might it struggle in large state spaces?
2. How does TD learning improve on direct evaluation by updating after every transition?
3. Why can't TD state-value learning, i.e., V(s), produce a policy without a model, and how do Q-values , i.e., Q(s,a), ix this?
4. Q-learning converges even when acting suboptimally. What assumptions must hold for this guarantee?
5. Explain the Q-learning update rule: how are old estimate, new sample, and learning rate combined?

Readings

123

- AIMA 21.1—21.5: Reinforcement Learning

What's next?

123

- Midterm Prep